

The order of growth of the running time of an algorithm, defined in Chapter 2, gives a simple way to characterize the algorithm's efficiency and also allows us to compare it with alternative algorithms. Once the input size  $n$  becomes large enough, merge sort, with its  $\Theta(n \lg n)$  worst-case running time, beats insertion sort, whose worst-case running time is  $\Theta(n^2)$ . Although we can sometimes determine the exact running time of an algorithm, as we did for insertion sort in Chapter 2, the extra precision is rarely worth the effort of computing it. For large enough inputs, the multiplicative constants and lower-order terms of an exact running time are dominated by the effects of the input size itself.

When we look at input sizes large enough to make relevant only the order of growth of the running time, we are studying the *asymptotic* efficiency of algorithms. That is, we are concerned with how the running time of an algorithm increases with the size of the input *in the limit*, as the size of the input increases without bound. Usually, an algorithm that is asymptotically more efficient is the best choice for all but very small inputs.

This chapter gives several standard methods for simplifying the asymptotic analysis of algorithms. The next section presents informally the three most commonly used types of “asymptotic notation,” of which we have already seen an example in  $\Theta$ -notation. It also shows one way to use these asymptotic notations to reason about the worst-case running time of insertion sort. Then we look at asymptotic notations more formally and present several notational conventions used throughout this book. The last section reviews the behavior of functions that commonly arise when analyzing algorithms.

### 3.1 $O$ -notation, $\Omega$ -notation, and $\Theta$ -notation

When we analyzed the worst-case running time of insertion sort in Chapter 2, we started with the complicated expression

$$\left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n - (c_2 + c_4 + c_5 + c_8).$$

We then discarded the lower-order terms  $(c_1 + c_2 + c_4 + c_5/2 - c_6/2 - c_7/2 + c_8)n$  and  $c_2 + c_4 + c_5 + c_8$ , and we also ignored the coefficient  $c_5/2 + c_6/2 + c_7/2$  of  $n^2$ . That left just the factor  $n^2$ , which we put into  $\Theta$ -notation as  $\Theta(n^2)$ . We use this style to characterize running times of algorithms: discard the lower-order terms and the coefficient of the leading term, and use a notation that focuses on the rate of growth of the running time.

$\Theta$ -notation is not the only such “asymptotic notation.” In this section, we’ll see other forms of asymptotic notation as well. We start with intuitive looks at these notations, revisiting insertion sort to see how we can apply them. In the next section, we’ll see the formal definitions of our asymptotic notations, along with conventions for using them.

Before we get into specifics, bear in mind that the asymptotic notations we’ll see are designed so that they characterize functions in general. It so happens that the functions we are most interested in denote the running times of algorithms. But asymptotic notation can apply to functions that characterize some other aspect of algorithms (the amount of space they use, for example), or even to functions that have nothing whatsoever to do with algorithms.

#### $O$ -notation

$O$ -notation characterizes an *upper bound* on the asymptotic behavior of a function. In other words, it says that a function grows *no faster* than a certain rate, based on the highest-order term. Consider, for example, the function  $7n^3 + 100n^2 - 20n + 6$ . Its highest-order term is  $7n^3$ , and so we say that this function’s rate of growth is  $n^3$ . Because this function grows no faster than  $n^3$ , we can write that it is  $O(n^3)$ . You might be surprised that we can also write that the function  $7n^3 + 100n^2 - 20n + 6$  is  $O(n^4)$ . Why? Because the function grows more slowly than  $n^4$ , we are correct in saying that it grows no faster. As you might have guessed, this function is also  $O(n^5)$ ,  $O(n^6)$ , and so on. More generally, it is  $O(n^c)$  for any constant  $c \geq 3$ .

**$\Omega$ -notation**

$\Omega$ -notation characterizes a *lower bound* on the asymptotic behavior of a function. In other words, it says that a function grows *at least as fast* as a certain rate, based—as in  $O$ -notation—on the highest-order term. Because the highest-order term in the function  $7n^3 + 100n^2 - 20n + 6$  grows at least as fast as  $n^3$ , this function is  $\Omega(n^3)$ . This function is also  $\Omega(n^2)$  and  $\Omega(n)$ . More generally, it is  $\Omega(n^c)$  for any constant  $c \leq 3$ .

 **$\Theta$ -notation**

$\Theta$ -notation characterizes a *tight bound* on the asymptotic behavior of a function. It says that a function grows *precisely* at a certain rate, based—once again—on the highest-order term. Put another way,  $\Theta$ -notation characterizes the rate of growth of the function to within a constant factor from above and to within a constant factor from below. These two constant factors need not be equal.

If you can show that a function is both  $O(f(n))$  and  $\Omega(f(n))$  for some function  $f(n)$ , then you have shown that the function is  $\Theta(f(n))$ . (The next section states this fact as a theorem.) For example, since the function  $7n^3 + 100n^2 - 20n + 6$  is both  $O(n^3)$  and  $\Omega(n^3)$ , it is also  $\Theta(n^3)$ .

**Example: Insertion sort**

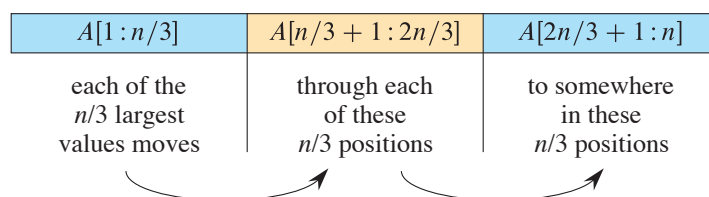
Let's revisit insertion sort and see how to work with asymptotic notation to characterize its  $\Theta(n^2)$  worst-case running time without evaluating summations as we did in Chapter 2. Here is the INSERTION-SORT procedure once again:

```

INSERTION-SORT( $A, n$ )
1  for  $i = 2$  to  $n$ 
2       $key = A[i]$ 
3      // Insert  $A[i]$  into the sorted subarray  $A[1 : i - 1]$ .
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 

```

What can we observe about how the pseudocode operates? The procedure has nested loops. The outer loop is a **for** loop that runs  $n - 1$  times, regardless of the values being sorted. The inner loop is a **while** loop, but the number of iterations it makes depends on the values being sorted. The loop variable  $j$  starts at  $i - 1$



**Figure 3.1** The  $\Omega(n^2)$  lower bound for insertion sort. If the first  $n/3$  positions contain the  $n/3$  largest values, each of these values must move through each of the middle  $n/3$  positions, one position at a time, to end up somewhere in the last  $n/3$  positions. Since each of  $n/3$  values moves through at least each of  $n/3$  positions, the time taken in this case is at least proportional to  $(n/3)(n/3) = n^2/9$ , or  $\Omega(n^2)$ .

and decreases by 1 in each iteration until either it reaches 0 or  $A[j] \leq \text{key}$ . For a given value of  $i$ , the **while** loop might iterate 0 times,  $i - 1$  times, or anywhere in between. The body of the **while** loop (lines 6–7) takes constant time per iteration of the **while** loop.

These observations suffice to deduce an  $O(n^2)$  running time for any case of INSERTION-SORT, giving us a blanket statement that covers all inputs. The running time is dominated by the inner loop. Because each of the  $n - 1$  iterations of the outer loop causes the inner loop to iterate at most  $i - 1$  times, and because  $i$  is at most  $n$ , the total number of iterations of the inner loop is at most  $(n - 1)(n - 1)$ , which is less than  $n^2$ . Since each iteration of the inner loop takes constant time, the total time spent in the inner loop is at most a constant times  $n^2$ , or  $O(n^2)$ .

With a little creativity, we can also see that the worst-case running time of INSERTION-SORT is  $\Omega(n^2)$ . By saying that the worst-case running time of an algorithm is  $\Omega(n^2)$ , we mean that for every input size  $n$  above a certain threshold, there is at least one input of size  $n$  for which the algorithm takes at least  $cn^2$  time, for some positive constant  $c$ . It does not necessarily mean that the algorithm takes at least  $cn^2$  time for all inputs.

Let's now see why the worst-case running time of INSERTION-SORT is  $\Omega(n^2)$ . For a value to end up to the right of where it started, it must have been moved in line 6. In fact, for a value to end up  $k$  positions to the right of where it started, line 6 must have executed  $k$  times. As Figure 3.1 shows, let's assume that  $n$  is a multiple of 3 so that we can divide the array  $A$  into groups of  $n/3$  positions. Suppose that in the input to INSERTION-SORT, the  $n/3$  largest values occupy the first  $n/3$  array positions  $A[1:n/3]$ . (It does not matter what relative order they have within the first  $n/3$  positions.) Once the array has been sorted, each of these  $n/3$  values ends up somewhere in the last  $n/3$  positions  $A[2n/3 + 1:n]$ . For that to happen, each of these  $n/3$  values must pass through each of the middle  $n/3$  positions  $A[n/3 + 1:2n/3]$ . Each of these  $n/3$  values passes through these middle

$n/3$  positions one position at a time, by at least  $n/3$  executions of line 6. Because at least  $n/3$  values have to pass through at least  $n/3$  positions, the time taken by INSERTION-SORT in the worst case is at least proportional to  $(n/3)(n/3) = n^2/9$ , which is  $\Omega(n^2)$ .

Because we have shown that INSERTION-SORT runs in  $O(n^2)$  time in all cases and that there is an input that makes it take  $\Omega(n^2)$  time, we can conclude that the worst-case running time of INSERTION-SORT is  $\Theta(n^2)$ . It does not matter that the constant factors for upper and lower bounds might differ. What matters is that we have characterized the worst-case running time to within constant factors (discounting lower-order terms). This argument does not show that INSERTION-SORT runs in  $\Theta(n^2)$  time in *all* cases. Indeed, we saw in Chapter 2 that the best-case running time is  $\Theta(n)$ .

## Exercises

### 3.1-1

Modify the lower-bound argument for insertion sort to handle input sizes that are not necessarily a multiple of 3.

### 3.1-2

Using reasoning similar to what we used for insertion sort, analyze the running time of the selection sort algorithm from Exercise 2.2-2.

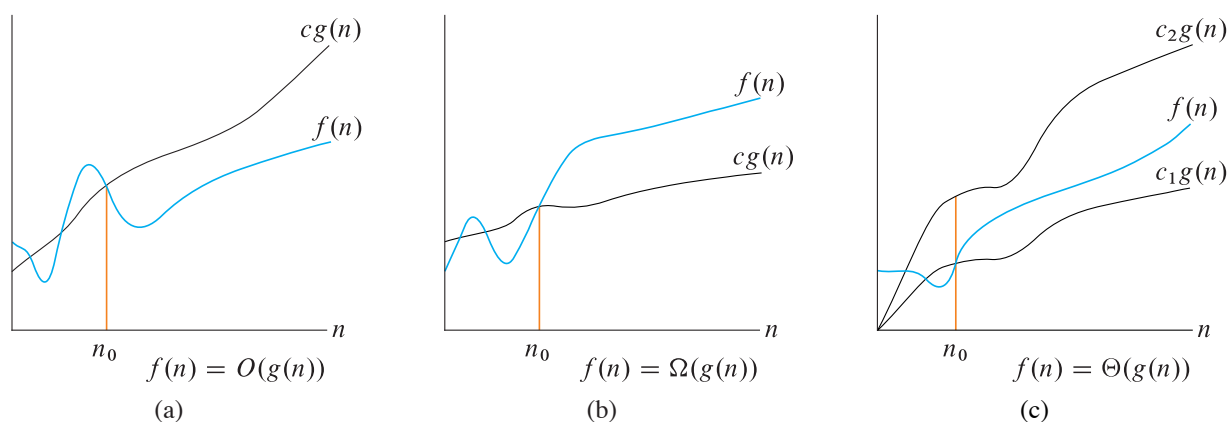
### 3.1-3

Suppose that  $\alpha$  is a fraction in the range  $0 < \alpha < 1$ . Show how to generalize the lower-bound argument for insertion sort to consider an input in which the  $\alpha n$  largest values start in the first  $\alpha n$  positions. What additional restriction do you need to put on  $\alpha$ ? What value of  $\alpha$  maximizes the number of times that the  $\alpha n$  largest values must pass through each of the middle  $(1 - 2\alpha)n$  array positions?

---

## 3.2 Asymptotic notation: formal definitions

Having seen asymptotic notation informally, let's get more formal. The notations we use to describe the asymptotic running time of an algorithm are defined in terms of functions whose domains are typically the set  $\mathbb{N}$  of natural numbers or the set  $\mathbb{R}$  of real numbers. Such notations are convenient for describing a running-time function  $T(n)$ . This section defines the basic asymptotic notations and also introduces some common “proper” notational abuses.



**Figure 3.2** Graphic examples of the  $O$ ,  $\Omega$ , and  $\Theta$  notations. In each part, the value of  $n_0$  shown is the minimum possible value, but any greater value also works. **(a)**  $O$ -notation gives an upper bound for a function to within a constant factor. We write  $f(n) = O(g(n))$  if there are positive constants  $n_0$  and  $c$  such that at and to the right of  $n_0$ , the value of  $f(n)$  always lies on or below  $cg(n)$ . **(b)**  $\Omega$ -notation gives a lower bound for a function to within a constant factor. We write  $f(n) = \Omega(g(n))$  if there are positive constants  $n_0$  and  $c$  such that at and to the right of  $n_0$ , the value of  $f(n)$  always lies on or above  $cg(n)$ . **(c)**  $\Theta$ -notation bounds a function to within constant factors. We write  $f(n) = \Theta(g(n))$  if there exist positive constants  $n_0$ ,  $c_1$ , and  $c_2$  such that at and to the right of  $n_0$ , the value of  $f(n)$  always lies between  $c_1g(n)$  and  $c_2g(n)$  inclusive.

### **$O$ -notation**

As we saw in Section 3.1,  $O$ -notation describes an *asymptotic upper bound*. We use  $O$ -notation to give an upper bound on a function, to within a constant factor.

Here is the formal definition of  $O$ -notation. For a given function  $g(n)$ , we denote by  $O(g(n))$  (pronounced “big-oh of  $g$  of  $n$ ” or sometimes just “oh of  $g$  of  $n$ ”) the *set of functions*

$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}^1$$

A function  $f(n)$  belongs to the set  $O(g(n))$  if there exists a positive constant  $c$  such that  $f(n) \leq cg(n)$  for sufficiently large  $n$ . Figure 3.2(a) shows the intuition behind  $O$ -notation. For all values  $n$  at and to the right of  $n_0$ , the value of the function  $f(n)$  is on or below  $cg(n)$ .

The definition of  $O(g(n))$  requires that every function  $f(n)$  in the set  $O(g(n))$  be *asymptotically nonnegative*:  $f(n)$  must be nonnegative whenever  $n$  is sufficiently large. (An *asymptotically positive* function is one that is positive for all

<sup>1</sup> Within set notation, a colon means “such that.”

sufficiently large  $n$ .) Consequently, the function  $g(n)$  itself must be asymptotically nonnegative, or else the set  $O(g(n))$  is empty. We therefore assume that every function used within  $O$ -notation is asymptotically nonnegative. This assumption holds for the other asymptotic notations defined in this chapter as well.

You might be surprised that we define  $O$ -notation in terms of sets. Indeed, you might expect that we would write “ $f(n) \in O(g(n))$ ” to indicate that  $f(n)$  belongs to the set  $O(g(n))$ . Instead, we usually write “ $f(n) = O(g(n))$ ” and say “ $f(n)$  is big-oh of  $g(n)$ ” to express the same notion. Although it may seem confusing at first to abuse equality in this way, we’ll see later in this section that doing so has its advantages.

Let’s explore an example of how to use the formal definition of  $O$ -notation to justify our practice of discarding lower-order terms and ignoring the constant coefficient of the highest-order term. We’ll show that  $4n^2 + 100n + 500 = O(n^2)$ , even though the lower-order terms have much larger coefficients than the leading term. We need to find positive constants  $c$  and  $n_0$  such that  $4n^2 + 100n + 500 \leq cn^2$  for all  $n \geq n_0$ . Dividing both sides by  $n^2$  gives  $4 + 100/n + 500/n^2 \leq c$ . This inequality is satisfied for many choices of  $c$  and  $n_0$ . For example, if we choose  $n_0 = 1$ , then this inequality holds for  $c = 604$ . If we choose  $n_0 = 10$ , then  $c = 19$  works, and choosing  $n_0 = 100$  allows us to use  $c = 5.05$ .

We can also use the formal definition of  $O$ -notation to show that the function  $n^3 - 100n^2$  does not belong to the set  $O(n^2)$ , even though the coefficient of  $n^2$  is a large negative number. If we had  $n^3 - 100n^2 = O(n^2)$ , then there would be positive constants  $c$  and  $n_0$  such that  $n^3 - 100n^2 \leq cn^2$  for all  $n \geq n_0$ . Again, we divide both sides by  $n^2$ , giving  $n - 100 \leq c$ . Regardless of what value we choose for the constant  $c$ , this inequality does not hold for any value of  $n > c + 100$ .

### $\Omega$ -notation

Just as  $O$ -notation provides an asymptotic *upper* bound on a function,  $\Omega$ -notation provides an *asymptotic lower bound*. For a given function  $g(n)$ , we denote by  $\Omega(g(n))$  (pronounced “big-omega of  $g$  of  $n$ ” or sometimes just “omega of  $g$  of  $n$ ”) the set of functions

$$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that} \\ 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}.$$

Figure 3.2(b) shows the intuition behind  $\Omega$ -notation. For all values  $n$  at or to the right of  $n_0$ , the value of  $f(n)$  is on or above  $cg(n)$ .

We’ve already shown that  $4n^2 + 100n + 500 = O(n^2)$ . Now let’s show that  $4n^2 + 100n + 500 = \Omega(n^2)$ . We need to find positive constants  $c$  and  $n_0$  such that  $4n^2 + 100n + 500 \geq cn^2$  for all  $n \geq n_0$ . As before, we divide both sides by  $n^2$ ,

giving  $4 + 100/n + 500/n^2 \geq c$ . This inequality holds when  $n_0$  is any positive integer and  $c = 4$ .

What if we had subtracted the lower-order terms from the  $4n^2$  term instead of adding them? What if we had a small coefficient for the  $n^2$  term? The function would still be  $\Omega(n^2)$ . For example, let's show that  $n^2/100 - 100n - 500 = \Omega(n^2)$ . Dividing by  $n^2$  gives  $1/100 - 100/n - 500/n^2 \geq c$ . We can choose any value for  $n_0$  that is at least 10,005 and find a positive value for  $c$ . For example, when  $n_0 = 10,005$ , we can choose  $c = 2.49 \times 10^{-9}$ . Yes, that's a tiny value for  $c$ , but it is positive. If we select a larger value for  $n_0$ , we can also increase  $c$ . For example, if  $n_0 = 100,000$ , then we can choose  $c = 0.0089$ . The higher the value of  $n_0$ , the closer to the coefficient  $1/100$  we can choose  $c$ .

### $\Theta$ -notation

We use  $\Theta$ -notation for *asymptotically tight bounds*. For a given function  $g(n)$ , we denote by  $\Theta(g(n))$  (“theta of  $g$  of  $n$ ”) the set of functions

$$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that} \\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\}.$$

Figure 3.2(c) shows the intuition behind  $\Theta$ -notation. For all values of  $n$  at and to the right of  $n_0$ , the value of  $f(n)$  lies at or above  $c_1 g(n)$  and at or below  $c_2 g(n)$ . In other words, for all  $n \geq n_0$ , the function  $f(n)$  is equal to  $g(n)$  to within constant factors.

The definitions of  $O$ -,  $\Omega$ -, and  $\Theta$ -notations lead to the following theorem, whose proof we leave as Exercise 3.2-4.

### **Theorem 3.1**

For any two functions  $f(n)$  and  $g(n)$ , we have  $f(n) = \Theta(g(n))$  if and only if  $f(n) = O(g(n))$  and  $f(n) = \Omega(g(n))$ . ■

We typically apply Theorem 3.1 to prove asymptotically tight bounds from asymptotic upper and lower bounds.

### Asymptotic notation and running times

When you use asymptotic notation to characterize an algorithm's running time, make sure that the asymptotic notation you use is as precise as possible without overstating which running time it applies to. Here are some examples of using asymptotic notation properly and improperly to characterize running times.

Let's start with insertion sort. We can correctly say that insertion sort's worst-case running time is  $O(n^2)$ ,  $\Omega(n^2)$ , and—due to Theorem 3.1— $\Theta(n^2)$ . Although

all three ways to characterize the worst-case running times are correct, the  $\Theta(n^2)$  bound is the most precise and hence the most preferred. We can also correctly say that insertion sort's best-case running time is  $O(n)$ ,  $\Omega(n)$ , and  $\Theta(n)$ , again with  $\Theta(n)$  the most precise and therefore the most preferred.

Here is what we *cannot* correctly say: insertion sort's running time is  $\Theta(n^2)$ . That is an overstatement because by omitting “worst-case” from the statement, we're left with a blanket statement covering all cases. The error here is that insertion sort does not run in  $\Theta(n^2)$  time in all cases since, as we've seen, it runs in  $\Theta(n)$  time in the best case. We can correctly say that insertion sort's running time is  $O(n^2)$ , however, because in all cases, its running time grows no faster than  $n^2$ . When we say  $O(n^2)$  instead of  $\Theta(n^2)$ , there is no problem in having cases whose running time grows more slowly than  $n^2$ . Likewise, we cannot correctly say that insertion sort's running time is  $\Theta(n)$ , but we can say that its running time is  $\Omega(n)$ .

How about merge sort? Since merge sort runs in  $\Theta(n \lg n)$  time in all cases, we can just say that its running time is  $\Theta(n \lg n)$  without specifying worst-case, best-case, or any other case.

People occasionally conflate  $O$ -notation with  $\Theta$ -notation by mistakenly using  $O$ -notation to indicate an asymptotically tight bound. They say things like “an  $O(n \lg n)$ -time algorithm runs faster than an  $O(n^2)$ -time algorithm.” Maybe it does, maybe it doesn't. Since  $O$ -notation denotes only an asymptotic upper bound, that so-called  $O(n^2)$ -time algorithm might actually run in  $\Theta(n)$  time. You should be careful to choose the appropriate asymptotic notation. If you want to indicate an asymptotically tight bound, use  $\Theta$ -notation.

We typically use asymptotic notation to provide the simplest and most precise bounds possible. For example, if an algorithm has a running time of  $3n^2 + 20n$  in all cases, we use asymptotic notation to write that its running time is  $\Theta(n^2)$ . Strictly speaking, we are also correct in writing that the running time is  $O(n^3)$  or  $\Theta(3n^2 + 20n)$ . Neither of these expressions is as useful as writing  $\Theta(n^2)$  in this case, however:  $O(n^3)$  is less precise than  $\Theta(n^2)$  if the running time is  $3n^2 + 20n$ , and  $\Theta(3n^2 + 20n)$  introduces complexity that obscures the order of growth. By writing the simplest and most precise bound, such as  $\Theta(n^2)$ , we can categorize and compare different algorithms. Throughout the book, you will see asymptotic running times that are almost always based on polynomials and logarithms: functions such as  $n$ ,  $n \lg^2 n$ ,  $n^2 \lg n$ , or  $n^{1/2}$ . You will also see some other functions, such as exponentials,  $\lg \lg n$ , and  $\lg^* n$  (see Section 3.3). It is usually fairly easy to compare the rates of growth of these functions. Problem 3-3 gives you good practice.

### Asymptotic notation in equations and inequalities

Although we formally define asymptotic notation in terms of sets, we use the equal sign ( $=$ ) instead of the set membership sign ( $\in$ ) within formulas. For example, we wrote that  $4n^2 + 100n + 500 = O(n^2)$ . We might also write  $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$ . How do we interpret such formulas?

When the asymptotic notation stands alone (that is, not within a larger formula) on the right-hand side of an equation (or inequality), as in  $4n^2 + 100n + 500 = O(n^2)$ , the equal sign means set membership:  $4n^2 + 100n + 500 \in O(n^2)$ . In general, however, when asymptotic notation appears in a formula, we interpret it as standing for some anonymous function that we do not care to name. For example, the formula  $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$  means that  $2n^2 + 3n + 1 = 2n^2 + f(n)$ , where  $f(n) \in \Theta(n)$ . In this case, we let  $f(n) = 3n + 1$ , which indeed belongs to  $\Theta(n)$ .

Using asymptotic notation in this manner can help eliminate inessential detail and clutter in an equation. For example, in Chapter 2 we expressed the worst-case running time of merge sort as the recurrence

$$T(n) = 2T(n/2) + \Theta(n) .$$

If we are interested only in the asymptotic behavior of  $T(n)$ , there is no point in specifying all the lower-order terms exactly, because they are all understood to be included in the anonymous function denoted by the term  $\Theta(n)$ .

The number of anonymous functions in an expression is understood to be equal to the number of times the asymptotic notation appears. For example, in the expression

$$\sum_{i=1}^n O(i) ,$$

there is only a single anonymous function (a function of  $i$ ). This expression is thus *not* the same as  $O(1) + O(2) + \cdots + O(n)$ , which doesn't really have a clean interpretation.

In some cases, asymptotic notation appears on the left-hand side of an equation, as in

$$2n^2 + \Theta(n) = \Theta(n^2) .$$

Interpret such equations using the following rule: *No matter how the anonymous functions are chosen on the left of the equal sign, there is a way to choose the anonymous functions on the right of the equal sign to make the equation valid.* Thus, our example means that for *any* function  $f(n) \in \Theta(n)$ , there is *some* function  $g(n) \in \Theta(n^2)$  such that  $2n^2 + f(n) = g(n)$  for all  $n$ . In other words, the right-hand side of an equation provides a coarser level of detail than the left-hand side.